

qresearch 技术手册

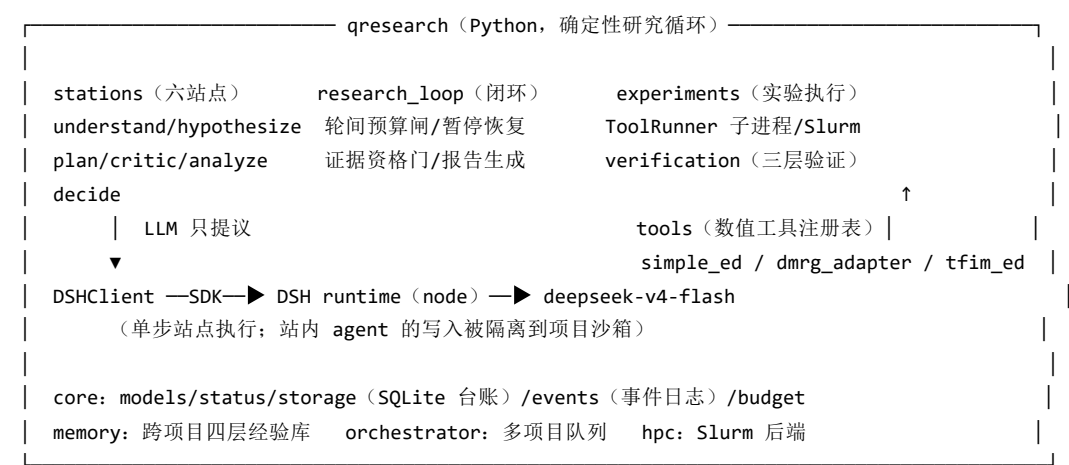
基于 DSH（DeepSeek Harness）的量子多体自主科研系统——完整技术参考。

面向使用者的操作指南与案例见 [user_guide.md](#)；阶段定义与验收标准见《基于DSH的量子多体自主科研系统开发计划_v2.md》§9。

1. 系统概览

qresearch 是一个**自主科研闭环系统**：给定一个物理研究问题，它自动完成"理解问题 → 提出假设 → 制定实验计划 → （人工审批）→ 执行数值实验 → 三层验证 → 分析证据 → 决策下一步"，循环往复直到给出结论或转人工。

拓扑：



角色分工：qresearch 主导流程、记账、校验、执行实验；DSH 只承担"单步站点"——每次调用给出一个受 JSON Schema 约束的输出（或一个编码交付物）。DSH 站内的 agent 可以自由思考、写草稿，但它的写入被限制在项目沙箱目录，它的输出必须通过 qresearch 的校验才能进入台账。

2. 五条铁律（贯穿全部实现）

#	铁律	实现机制
1	LLM 只提议，代码记账	所有状态写入（Storage/EventLog）只在确定性代码路径发生；LLM 输出仅在校验通过后由代码落账
2	实验不经 LLM	实验由 ExperimentManager 经 ToolRunner（默认子进程）执行注册工具；LLM 只能提议 inputs，永远接触不到执行环节
3	出题人≠答题人	工具 golden 基准由出题侧（Spec）锁定，数值来自解析/文献/独立实现；构建者交付的代码被同一套基准检验（详见 §9）

#	铁律	实现机制
4	终止/ 宣布结论无条件转人工	declare_result 决策一律 requires_human=True （模型层校验，代码层强制）；轮数上限处的 iterate 同样强制转人工（[预算闸] 留痕）
5	全程可回放	每个动作都是 Event，追加写入 events.jsonl；任何台账对象都能通过事件链回放其来历

安全不变量：DEEPSEEK_API_KEY 永不打印/落日志；auto_approve=True 时审批事件以 actor=system 留痕并注明"auto-approve（演示/测试用，非人工）"，绝不冒充人工。

3. 目录结构与模块地图

```
qresearch/
├─ core/                # 基础层
│   └─ models.py        # 13 个台账对象（pydantic）：Project/Goal/Hypothesis/
│                       #   PlanStep/ResearchPlan/Experiment/Evidence/CheckItem/
│                       #   Decision/ToolRecord/VerificationReportItem/
│                       #   VerificationReport/ProjectBundle
│   └─ status.py        # 全部状态机枚举（StrEnum）
│   └─ storage.py        # SQLite 台账：类型化 save/list/get + 引用完整性
│   └─ events.py        # EventLog：JSONL 追加 + 按项目过滤 + 线程锁
│   └─ budget.py        # Budget(max_rounds/max_experiments/wallclock_min)
│   └─ io.py            # 项目 bundle 导出/导入
│   └─ testing.py       # 演示数据工厂
├─ dsh_client.py        # DSH SDK 唯一封装点：call_station / run_agent / 看门狗
├─ stations/           # 七站点：schemas（pydantic输出）/ prompts / executors
├─ loop.py              # 规划阶段（understand→...→approve，含 revise 循环）
├─ research_loop.py     # 研究闭环：run_research_loop / resume_research_loop
├─ experiments/
│   └─ manager.py       # ExperimentManager + ToolRunner 协议 + 并行
├─ tools/              # 数值工具：registry（ToolSpec）+ simple_ed/dmrg_adapter/tfim_ed + cli
├─ verification/       # golden.py（基准执行）+ manager.py（三层验证 + 证据资格门）
├─ tool_builder/       # spec.py（Spec 加载）+ builder.py（构建/修复/审查/晋升）
├─ memory/             # store.py（四层经验库）/ distill.py（确定性蒸馏）/ inject.py（检索注入）
├─ hpc/slurm.py        # SlurmConfig / render_sbatch / SlurmRunner(ToolRunner)
├─ viz.py              # plot_project：台账→三张 PNG（可选依赖 matplotlib）
├─ ui/                 # 人机交互层（Phase 9）：progress.py（事件驱动进度条）/
│                       #   plan_doc.py（计划 markdown 文档）/ session.py（CLI 会话）
├─ orchestrator.py     # ProjectJob + run_projects（多项目队列）
├─ tool_specs/         # 工具规格（出题侧）：tfim_ed.yaml + golden 定稿
├─ benchmarks/golden/  # 安装后的 golden 基准（simple_ed / dmrg_vs_simple_ed / tfim_ed）
├─ examples/          # phase2-8 验收演示 + smoke test + resume 演示
├─ tests/              # 104 项 pytest
├─ research_data/      # 运行数据（gitignore）：每项目独立目录
```

4. core：台账与事件

4.1 对象模型（models.py）

对象	关键字段	语义
Project	project_id, question, status	研究项目
Goal	refined_question, quantities, success_criteria	understand 站点的产物
Hypothesis	statement, rationale, falsification_tests, discriminating_experiment, status	模型层规则： 必须带证伪试验
PlanStep	action, purpose, tools, inputs, expected_outputs	action ∈ run_experiment / parameter_scan
ResearchPlan	version, steps, risks, based_on_decision	版本递增；replan 挂链决策 id
Experiment	experiment_id, tool_id, parameters, status, artifacts, verification_status	artifacts[0]=result.json，artifacts[1..]=脚本等
Evidence	claim, experiment_ids, source, status	只能引用存在的实验；资格门决定 status
CheckItem	claim, status, evidence	决策检查项：passed 项 必须挂 evidence
Decision	type, recommendation, checklist, requires_human	declare_result 无条件 requires_human
ToolRecord	name, version, input_schema, benchmark_refs, known_limitations	工具注册的台账投影
VerificationReport	overall, items	三层验证结果
ProjectBundle	全对象聚合	导出/导入 roundtrip

4.2 状态机（status.py）

- ExperimentStatus : created → running → completed / failed / cancelled
- VerificationStatus : not_run / passed / failed / uncertain
- PlanStatus : draft → awaiting_approval → approved → executing → completed / superseded / rejected
- HypothesisStatus : proposed → under_test → supported / weakened / rejected / accepted_provisionally
- DecisionRecommendation : iterate / replan / terminate / declare_result（建议）； DecisionType : iterate_or_terminate / declare_result

4.3 Storage（storage.py）

类型化 SQLite： storage.save(obj) / storage.list(Model, project_id=...) / storage.get(Model, id)。表结构由模型注册表驱动；project_id 列保证多项目隔离。所有跨对象引用（evidence→experiment、plan→decision、checklist→evidence）在 load_bundle 时校验完整性。

4.4 EventLog（events.py）

JSONL 追加（一事件一行），字段： timestamp / actor(model|system|human) / action / project_id / object_type / object_id / detail。自带线程锁（并行实验时安全）。

subscribe(callback)：追加时同步回调事件——显示层（ui.progress）借此实时渲染。订阅者异常被吞掉（铁律：显示是视图，绝不干扰记账闭环）。

回放：EventLog(path).events(project_id=...) 按序返回——这是审计与调试的第一入口。常见事件：

事件	时机
create_project / save	项目与台账对象写入
station_call / station_retry	站点轮次成功 / runtime 层失败重试
critique / plan_ready / approve	critic 审查 / 计划就绪 / 审批（auto_approve 时 actor=system）
experiment_started / experiment_finished	实验起止（并行时 started 全部先于任何 finished——记账在主线程）
verify_experiment	三层验证完成
analyze / decide	轮内分析 / 决策
memory_retrieved / memory_written	记忆检索注入 / 项目结束蒸馏入库
budget_exhausted / paused / resumed	预算闸 / 暂停 / 恢复
needs_human / report_generated	优雅转人工 / 报告生成

4.5 Budget（budget.py）

Budget(max_rounds=None, max_experiments=None, wallclock_min=None)。

exhausted(rounds_used=, experiments_used=, elapsed_s=) 返回耗尽原因字符串（如 "实验数预算耗尽（2/2）"）或 None。**代码层轮间硬闸**：在每轮开始前检查，LLM 不可越过；耗尽原因写入事件与 summary。

5. dsh_client：DSH 封装层

DSHClient 是系统与 DSH runtime 的唯一接口（换 runtime 只改此文件）。

```
DSHClient(dsh_home=..., cwd=..., model=None, runner=None,
          station_timeout_s=None, **harness_kwargs)
```

- runner 注入后完全离线（测试/演示）：(prompt, session_id) -> str。
- cwd：DSH 站内 agent 的工作目录——**研究项目应指向项目沙箱**，站内写入被隔离。
- station_timeout_s：站点轮次 wallclock 上限（默认 900s；环境变量 QRESEARCH_STATION_TIMEOUT_S 可覆盖）。

5.1 call_station：校验流水线

```
call_station(station, project_id, schema, prompt, retries=1, event_log=None, validator=None)
```

执行顺序：prompt + JSON Schema 指令 → SDK 调用 → extract_json（容忍围栏/散文）→ pydantic schema 校验 → 可选 validator（语义校验，如"引用的实验必须存在"）。

失败带错误反馈重试（_RETRY_INSTRUCTION），按 a{attempt} 换 session；

runtime 层失败（超时/断链/协议错误）同样换 session 重试并落账 station_retry

事件；重试耗尽抛 NeedsHuman(station, last_output, reason) ——闭环在轮边界优雅转人工，不炸整个项目。

5.2 看门狗（StationTimeout）

SDK 的会话等待没有超时参数；runtime 子进程若中途死亡，等待可能永不返回（P8 live 实测悬挂 ~21 分钟才由 runtime 侧兜底）。_run 因此把 SDK 调用放进 worker 线程并 join(timeout)：超时 → _restart_harness()（关闭旧 runtime、按原参数重启，旧 RPC 随旧进程消亡）→ 抛 StationTimeout。这把"可能永远卡死"变成有界、可重试、可落账的确定性失败。

5.3 run_agent：开放轮次

无 schema 约束的 agent 任务（Tool Builder 编码站用）：交付物是**文件**而非结构化输出，由调用方在文件层面验收。同样受看门狗保护。

6. stations：七个 LLM 站点

每个站点 = 一个 prompt 模板（prompts.py）+ 一个输出 schema（schemas.py）+ 一个执行器（executors.py，负责调用 call_station、落账、返回台账对象）。

站点	输出 schema	要点
understand	UnderstandOutput	精炼问题 refined_question、目标量 quantities、成功判据 success_criteria；问题不可解析时 诚实自报 constraints.blocking=true + requires_clarification 清单（不虚构物理系统），引擎确定性闸门在假设/计划之前拦下并转人工（2026-09-15：闭环不读该信号就会机械空转的实测教训）
hypothesize	HypothesizeOutput	n 条假设， 每条必须带 falsification_tests 与 discriminating_experiment （模型层拒绝缺证伪试验的假设）
plan	PlanOutput	steps（action/tools/inputs/expected_outputs）+ risks + diff_summary（修订版必须逐条说明改动）；语义 validator：tools 必须引用已注册工具、可执行动作恰好挂 1 个工具、parameter_scan 必须带 inputs.scan={参数名:[取值...]}；模板含动作选择指引与 记忆注入块 {memory}
critic	CritiqueOutput	verdict pass/blocker + issues；对计划做独立物理审查
analyze	AnalyzeOutput	observations（ 实验引用必须是本轮已验证实验 ，资格门在下游兜底）/ interpretations / uncertainties / recommended_next_steps
decide	DecideOutput	recommendation（iterate/replan/terminate/declare_result）+ checklist（passed 项必须挂 evidence id——模型无法引用看不见的 id，prompt 显式列出）+ info_gain_estimate；模板含 记忆注入块 （失败案例优先）
transcribe	PlanOutput	编辑通道（Phase 9）：把用户直接编辑的计划文档忠实转录回结构化计划——不许增删或"纠正"用户编辑；产物仍过同一套语义 validator + critic（用户编辑不绕过验证管线）

审批触点：plan 通过 critic 后进入 awaiting_approval；人工（或 auto_approve=True 下的 system 留痕）批准后 plan_ready → approve。loop.run_planning_phase 提供 revise 循环（critic blocker → 修订版计划 → 重申）。

7. experiments：实验执行

7.1 ExperimentManager

```
ExperimentManager(storage, event_log, runner=None, # 缺省 SubprocessToolRunner
                  experiments_root=None, # 缺省 research_data/experiments
                  default_timeout_s=900.0)
```

- `execute_plan(plan, include_requires_approval=False, max_workers=1)`：
逐 step 展开为 Experiment（parameter_scan 网格展开为单点实验），全部 **prepare**（started 事件落账）后进入执行，完成即 `finalize`。
- `execute_scan(plan, step)`：单步扫描的公开入口。

7.2 并行模型（Phase 8）

只并行**计算段**： `max_workers>1` 时实验在 `ThreadPoolExecutor` 中跑 runner 调用；所有台账写入（`finalize`/事件）保持在主线程按提交顺序执行——
Storage（SQLite）非线程安全，EventLog 自带锁。可观测不变量：事件日志里 `experiment_started` 全部先于任何 `experiment_finished`。
每个实验的 per-job 计时（`started_at/finished_at`）按各自实际起止记录。

7.3 ToolRunner 协议

```
class ToolRunner: # 协议
    def run(self, spec: ToolSpec, inputs: dict, workspace: Path, timeout_s: float) -> RunOutcome: ...
```

- `SubprocessToolRunner`（默认）：`python -m qresearch.tools.cli run --tool {name} --inputs-file inputs.json --out result.json`，子进程退出码非零 → `FAILED`。
工具崩溃/超时不经 LLM、不污染台账（错误文本落账）。
- `SlurmRunner`（hpc）：见 §12。
- 注入自定义 runner 即可换执行后端（测试中用假 runner 验证并行语义）。

实验产物：`workspace/inputs.json`、`result.json`、`job.sbatch`（Slurm 时）等，
路径记录在 `Experiment.artifacts`。

8. tools：数值工具与注册表

8.1 ToolSpec 与 registry

```
@dataclass
class ToolSpec:
    name: str; version: str; type: str; description: str
    input_model: type[BaseModel]          # pydantic 输入模型（JSON Schema 供 LLM）
    run: Callable[[dict], dict]            # 已校验输入 → 结果 dict
    applicable_domains: list[str]           # 供 PLAN 检索的适用域标签
    benchmark_refs: list[str]              # golden 基准文件（出题侧锁定）
    known_limitations: list[str]           # 如实声明的限制

register(spec, replace=False) / get_tool(name) / unregister(name) / tool_names()
load_seed_tools()  # 导入 seed 工具模块完成注册（ExperimentManager 初始化时自动调用）
```

replace=True 仅供 Tool Builder 晋升/重建升级路径使用（同名覆盖）。

8.2 种子工具

工具	物理内容	备注
simple_ed	自旋 1/2 Heisenberg 链 ED（PBC，Sz=0 扇区，稠密/稀疏双后端）+ 关联函数	N≤14 稠密可用；ARPACK 固定 v0 保证可复现
dmrg_adapter	quimb DMRG2，失败自动回退 MPO 稠密对角化	已知平台缺陷： quimb 1.15 局部求值器本机产生 NaN（method / fallback_note 留痕）；P8 live 中该缺陷产生 29 条失败案例入库
tfim_ed	横场 Ising ED（Lanczos/稠密双后端，E0+Rayleigh 残差+能隙，含简并判据）	Phase 6 由 DSH 构建 ，golden 8/8 首轮通过后晋升入 seed

8.3 工具 CLI（子进程入口）

```
python -m qresearch.tools.cli run --tool simple_ed \
    --inputs experiments/<id>/inputs.json --out experiments/<id>/result.json
```

加载 seed 注册表 → 取 ToolSpec → 读输入 → spec.run(inputs) → 写 JSON。
任何异常打印 TOOL RUN FAILED: 类型：消息 到 stderr 并返回码 1。

8.4 golden 基准（benchmarks/golden/*.yaml）

出题侧锁定的断言文件：解析值（如 Bethe ansatz $e_0=1/4-\ln 2$ 、TFIM 临界 $e_0(\infty)=-4J/\pi$ ）、独立实现差分（dmrg vs simple_ed）、热力学极限、自洽性。
新增工具必须先出 Spec 与 golden，再允许实现 (§9)。

9. verification：三层验证与证据资格门

9.1 三层验证

VerificationManager(storage, event_log).verify_experiment(exp, tool_name) -> VerificationReport
跑该工具的全部 golden 检查并按层裁决：

层	内容
software 程序层	同参数复跑一致（物理精度内可重复，相对容差 1e-10）、结果字段完备、边界 case
numerical 算法层	收敛残差 / 能量方差 / 变分上界等数值不变量（输出缺失判 uncertain，不放过）
physics 物理层	golden 基准：解析值 / 独立实现差分 / 热力学极限 / 自治性连锁

某层无检查项 → 该层 UNCERTAIN（"无覆盖"本身被记录，不默认放行）。
overall ∈ {passed, failed, uncertain}。

9.2 证据资格门

ANALYZE 之后、证据定 status 时执行：只有 overall=passed 的实验可被引用为 passed 证据；FAILED 实验与验证未通过的实验只能以"失败/不确定"身份进入分析（失败本身是信息）。dry-run 结果（无工具输出字段）自然过不了三层，资格门自动拒绝——诚实失败优于冒充成功。

10. tool_builder：工具自动构建（Spec 先行 + 防串通）

流程
(
build_tool(client, storage, event_log, spec_path, max_repair_rounds=3, auto_approve=False, builds_root=None, client_factory=None)
):

1. **Spec 先行**：tool_specs/<tool>.yaml（出题侧）定义目的/输入模型/物理断言/ golden 定稿（数值来自解析/文献/独立实现，**编码 prompt 零基准数值**——测试断言保证）。
2. **编码站**：DSH run_agent（client_factory(workspace) 每 attempt 以 cwd=workspace 新建 runtime，agent 的交付目录就是工作目录）；交付 tool_module.py（含 pydantic Inputs + run）+ selftest.py（要求独立 oracle、显式声明"未使用基准常数"）。
3. **golden 验证**：交付实现跑出题侧 golden——失败进入修复轮（错误反馈），最多 max_repair_rounds 次。
4. **批评者审查**：独立 critic 站审 diff（物理正确性/边界/ limitation），P6 live 中它曾用独立参考值复核 gap 归零争议（8.56e-12 vs 8.57e-12）。
5. **人工审批**（铁律触点）→ register(spec, replace=True) 注册。
6. **晋升**（人工/代行审查后）：交付代码**逐字节**复制入 qresearch/tools/（仅追加 seed 注册块，diff-vs-Spec 可追溯），golden fixtures 随晋升提交。

已完成的晋升案例：tfim_ed（首轮过 golden 8/8；构建者以全枚举+解析论证反驳了 Spec 初版"奇数 N 铁磁受挫"的错误表述，出题侧采纳修订——出题人≠答题人的双向实例）。

11. research_loop：研究闭环

11.1 主流程

```
run_research_loop(client, storage, event_log, project_id, question, *,
                  rounds=3, auto_approve=False, n_hypotheses=3, retries=1,
                  experiments_root=None, memory_store=None, budget=None,
                  pause_flag=None, max_parallel_experiments=1,
                  round_callback=None) -> dict
```

Round 0: understand → hypothesize (失败转 needs_human)。

每轮 (1..rounds):

轮开始 →▶ 暂停检查 (pause_flag 存在 → paused, 等待 resume)
→▶ 预算检查 (Budget 耗尽 → budget_exhausted + reason)
→▶ plan_with_critic (记忆注入 + 用户修改意见注入; critic blocker → 修订)
→▶ 审批 (对话式人工 / auto_approve 留痕)
→▶ execute_plan (实验可并行) → verify (三层) → analyze (资格门)
→▶ decide (记忆注入; declare_result/terminate → 收束;
 末轮 iterate → [预算闸] 强制 requires_human)
→▶ round_callback (仅"还要继续"的轮次; None/"stop"/意见字符串)

结束路径: terminated (decide 收束) / budget_exhausted (含 reason) /

paused / stopped_by_user / {"needs_human": 轮次:原因}。 _finalize

生成 report.md (置顶研究总结 + 计划版本历史 + 假设/实验表/分析结论/

决策记录/局限) 并蒸馏记忆入库。

11.2 人工交互：对话式审批与轮末回调

对话式计划审批 (loop._interactive_approval): 每版计划先由

ui.plan_doc.save_plan_doc 渲染成 <项目目录>/plans/plan_vN.md (确定性

渲染, LLM 不参与; 文末带"修改意见"节与编辑说明), 展示摘要后循环

[y]批准 / [c]提修改意见 / [e]编辑文档 / [s]看步骤细节 / [q]放弃。

- 意见通道: c (多行, 空行结束) 或**直接输入一段文字** → plan_feedback
事件 (actor=HUMAN, mode=verbal) → plan_with_critic(previous=当前版, user_notes=意见) 生成新版本 (重新过 critic、重新等审批);
- 编辑通道: e → 用户改文档 → 回车 → split_user_notes 拆正文/意见 →
transcribe 站点转录 (mode=edit_doc 留痕) → 同一语义 validator →
critic 独立重申 → 新版本。 **用户编辑不绕过验证管线。**
EOF/Ctrl+C 一律按 q (放弃) —— **绝不默认批准。** user_notes 在计划 prompt 的"研究者本人的修改意见"栏以最高优先级注入; 模型不采纳必须写明理由。修订次数受 max_revisions (默认 5) 约束。
返回最终计划 (调用方以返回值为准继续 EXECUTE)。

轮末回调 (round_callback, fresh/resume 均支持): 每轮 decide 后 (且

本轮未收束时) 以 dict 摘要调用—— round_no / decision (类型/建议/

rationale) / experiments_this_round / evidence_this_round / evidence_total。

返回 "stop" → user_stop 事件 + stopped_by_user 终态 (可 resume);

返回非空字符串 → user_feedback 事件 (actor=HUMAN) + 注入下一版计划

(_LoopState.user_notes , 只生效一轮后清空)。收束轮不触发——结论走

declare_result 无条件人工确认通道。GUI/服务可据此实现节点推送与随时插话。

11.3 报告生成 (generate_report)

确定性拼装，无 LLM 参与：**研究总结**（最终状态+决策链；结论=收束决策 rationale 原文+决策 id；支撑证据=declare/accept 决策 checklist 中 passed 项逐条挂 evidence id；假设命运=支持/反驳/未判定；规模统计）→ 假设 → **计划版本历史**（每版：状态/步骤数、依据决策、diff_summary、critic 判定与阻塞项、审批者——由 _finalize 从 plan_ready/approve 事件重建 plan_notes）→ 实验表 → 分析结论 → 决策记录 → 局限。报告路径：state.sqlite 旁 report.md。

11.4 可视化 (viz, 可选依赖)

plot_project(storage, project_id, out_dir) -> list[Path] 三张 PNG：
工具×状态堆叠柱图；数值结果 vs 参数（按计划版本着色，实心=passed/空心=未过验证；只画“有变化的参数”≥2 个不同取值，排除 _ 前缀簿记参数与结果中回显的输入值）；证据累计曲线+决策时间线。matplotlib 延迟导入（Agg 后端、中文字体 Microsoft YaHei/SimHei），缺失时报错提示 pip install -e ".[viz]"。全部数据来自台账——含失败、不做美化（诚实边界的可视化表述）。

11.5 交互层 (Phase 9, qresearch.ui)

职责边界：**视图与控制**——零研究决策、零台账写入；所有用户编辑走“转录→语义校验→critic→审批”管线。三个组件：

- **progress.py** ConsoleProgress(event_log)：订阅 EventLog，rich Live 单行刷新 spinner + 统计（第 R 轮/实验完成失败/运行中/验证/已用时间），station_started 事件由 dsh_client 每次尝试前发出；里程碑事件（plan_ready/approve/decide/...）持久打印不刷掉。rich 缺失时退化为单行刷新。订阅者异常被 EventLog 吞掉，显示永不打断闭环。
- **plan_doc.py**：render_plan_markdown（台账对象→markdown，含步骤/输入yaml/预期输出/风险/diff/critic 结论与 blocker 标记）+ save_plan_doc（写 plans/plan_vN.md）+ split_user_notes（编辑后文档拆正文与“修改意见”节）。
- **session.py** qresearch 命令（[project.scripts] 入口，[ui] extra）：
REPL 会话——自然语言问题确认后调 run_research_loop(auto_approve=False, round_callback=...)；斜杠命令 /projects /status /report /plots /pause /resume /quit；轮末交互点自由输入=意见、stop=叫停；Ctrl+C 一次=轮末优雅停（stop_requested → 回调返回 "stop"），两次=立即中断（台账安全，可 resume）。
单线程 + 终端 typeahead：实验执行期间敲的字在下一个交互点被读，无需并发。
会话只把你的话路由成**参数/反馈/命令**，研究决策仍在七站点闭环里。

11.6 恢复 (真相在库里)

```
resume_research_loop(client, storage, event_log, project_id, *, rounds=3, ...)
```

从台账重建 Goal/假设/最新计划/决策链/证据/工具映射，start_round=len(decisions)+1，fresh 与 resume 共享同一 _rounds_loop 主干（无双实现漂移）。适用：暂停续跑、进程中断恢复。计划版本跨恢复衔接（实测 [1,2,3]）。

12. memory：跨项目研究记忆（Phase 7）

12.1 四层经验库

MemoryStore(db_path, markdown_path=None)（独立 SQLite + Markdown 镜像供人审）：

add / get / list(layer) / search(query, layers=, limit=) / close。

层：project（问题+假设命运+决策轨迹）/ method（被验证支撑的方法主张，上限 6）/

tool（使用/失败/验证统计）/ failure（FAILED 实验 + 无信息增益实验，含错误与教训）。

全部条目携带问题域 tags 与人类可读工具名。

12.2 确定性蒸馏

distill_project(storage, project_id)：项目结束时**代码**从台账提取（不经 LLM），

每条记忆可回放回到源实验/验证报告/决策——记忆本身不会成为无来源结论。

一实验一条失败记录（FAILED 或 完成但未通过验证，if/elif 保证不重复）。

12.3 检索注入

memory_digest(store, query, limit=10) -> (text, n_hits)：确定性关键词打分

（拉丁分词 + 中文 2-gram，零分不返回），**失败案例区排最前**。每轮注入

PLAN 与 DECIDE prompt；模板明确"与拟议步骤雷同的失败案例必须先修正做法或

写明差异"。事件：memory_retrieved（每轮）/ memory_written（项目结束）。

不接 memory_store 时闭环行为不变（占位"（无）"）。

13. hpc + orchestrator：HPC 后端与多项目队列（Phase 8）

13.1 Slurm 后端

```
SlurmConfig(partition="cpu", time_min=60, mem_gb=4, cpus=1, account=None,
             python_exe="python", submit_cmd="sbatch", poll_interval_s=10)
render_sbatch(tool_name, inputs_file, out_file, *, cfg, job_name=None) -> str # 纯函数
SlurmRunner(cfg, dry_run=True) # ToolRunner 实现
```

- dry_run=True（默认）：只生成并校验 sbatch 脚本（#SBATCH 分区/时限/内存/CPU/账户 + python -m qresearch.tools.cli run ...），**不提交**。dry-run 结果带 dry_run 标记且无工具输出 → 三层验证不放行、证据门拒绝（诚实边界落在机制上）。
- 真实路径（需集群验收）：sbatch 提交 → 解析 job id → squeue 轮询 → result.json 回收。
- 接入方式：ExperimentManager(runner=SlurmRunner(cfg)) 一行切换；实验不经 LLM 的铁律不变。submit_cmd 可换成 ["ssh", "user@cluster", "sbatch"] 形态跨机提交。

13.2 多项目编排

```
@dataclass
class ProjectJob:
    project_id: str; question: str; rounds: int = 3
    max_parallel_experiments: int = 1; budget: Budget | None = None
    n_hypotheses: int = 3; extra: dict = ...

run_projects(client_factory, jobs, *, data_root, memory_store=None,
             retries=1, auto_approve=False) -> {project_id: summary}
```

队列语义：项目**串行**（LLM 站点是瓶颈，串行即可饱和）；项目内实验按 `max_parallel_experiments` 并行。每项目独立目录布局：

```
<data_root>/<project_id>/
├─ state.sqlite      # 台账
├─ events.jsonl      # 事件日志
├─ sandbox/          # DSH 站内 agent 写入沙箱（client cwd）
├─ experiments/      # 实验产物（或全局 research_data/experiments）
├─ report.md         # 自动报告
└─ PAUSE             # 暂停旗标（写入即在轮间暂停）
```

`client_factory(sandbox)` 是运行时注入点：真实 LLM 传
`lambda sb: DSHClient(cwd=sb, dsh_home=...)`（每项目独立 runtime）；离线测试
传脚本客户端。 `auto_approve` 默认 `False`——长期队列不默认越过人工审批。

14. engine + mcp_server：接口化内核与 MCP 接入（计划 v3）

计划 v3（分支 `feature/interactive-agent`）把研究闭环包成两层对外接口：

`ResearchEngine`（Python 门面，M1）与 `qresearch/mcp_server.py`（MCP server，M2）。五条铁律全部保留，其中铁律 3 在接口层落成 **A1 审批台账仲裁**：
MCP 工具面**没有任何**能写“批准/结论”的工具（也无 `declare_result`，D6）——
“写”只存在于真实终端的 CLI 审批台，且带 TTY 守卫（无人到场批不了）。

14.1 ResearchEngine 门面（qresearch/engine.py，M1）

```
engine = ResearchEngine(storage, event_log, client, *,
                        experiments_root=..., runner=None, budget=None,
                        retries=1, memory_store=None)

engine.open_project(pid, question) -> Project      # 幂等（台账已有即返回）
engine.state(pid) / engine.status(pid) -> dict      # 状态投影（供 UI/MCP）
engine.understand(pid, question) / engine.hypothesize(pid, n=3)
engine.plan_create(pid, *, round_no=..., user_notes=...) -> (plan, critique)
engine.plan_revise(pid, plan_id, user_notes)        # 版本自动递增
engine.plan_approve(pid, plan_id, *, actor, note)    # A1: MCP 面不暴露 actor 写入
engine.experiments_run(plan_id, *, max_workers=1, cancel_check=None) -> dict
engine.experiments_run_async(plan_id) -> job_id      # A2: MCP 层专用
engine.verify(ids=None) / engine.analyze(pid, experiment_ids=...) / engine.decide(pid)
engine.report_generate(pid, status=None) / engine.viz_plot(pid, out_dir)
engine.adhoc_record / engine.adhoc_verify           # A6: 计划外记账纪律
```

原则（A3）：**内核方法全部同步**——事件时序与 v2 严格一致（L1 基线锁定）；
异步（job 协议）只存在于 MCP 一层（experiments_run_async 是唯一的例外入口，
内部就是 JobManager.submit(key="experiments:<plan_id>") ）。

14.2 JobManager（qresearch/jobs.py，A2+A3）

进程内异步 job 注册表，**只服务 MCP 层**（壳 B 的闭环不经过它）：

```
jobs.submit(fn, *args, key=None, total=None) -> job_id    # 同 key 活跃 job 幂等（D4）
jobs.status(job_id) -> {state, cancel_requested, done, total, last_event, elapsed_s}
jobs.result(job_id) -> {state, result | error}              # 未完成时给轮询提示
jobs.cancel(job_id) -> bool                                # pending→True; running→置请求返回 False
jobs.cancel_requested(job_id) / jobs.find_active(key)
```

状态机 pending → running → done | error，另 pending/running → cancelled。
job 生命周期是**进程内状态，不落台账**；台账事件由 job 内执行的引擎方法照常
落账——异步化不改变台账语义（A5 归一化比对验证过）。MCP 工具调用默认 60s
超时，同步返回必超时、agent 会误判重跑产生重复实验（D4）——所以**一切长工具**
（**站点调用与实验 alike**）都返回 job_id，调用方轮询。

14.3 MCP server（qresearch/mcp_server.py，M2，22 工具）

stdio 传输；python -X utf8 qresearch/mcp_server.py --project-root <root> --model <m>。
server 内嵌自己的 DSHClient（A7，懒创建：首个需要 LLM 的工具触发，
cwd=<project>/sandbox）。工具面：

类别	工具
项目	research_open / research_status / research_list
站点（长，A2）	understand / hypothesize / plan_create / plan_revise
计划（读）	plan_show（含 markdown 卡片） / plan_approve（ 纯查询 ）
实验（长）	experiments_run（计划须 APPROVED） / adhoc_record / adhoc_verify
分析链	verify / analyze（PASSED 才有资格） / decide（requires_human 无条件）
产出	report_generate / viz_plot
台账	ledger_query / events_tail
job 协议	job_status / job_result / job_cancel

- **A1 仲裁**：plan_approve(project_id, plan_id) 只查台账里是否有 actor=HUMAN 的 approve 事件，返回 approved_by_human 与 how_to（CLI 指引）。写路径唯一入口：qresearch approve|reject <dir> <plan_id>。
- **D6**：decide 的 declare_result/terminate 一律 requires_human=true；
确认只经 qresearch conclude <dir> <decision_id>（幂等）。
- 错误形状：异常不穿透——工具返回 {"error": "<类型>: <消息>" }，agent 可读。

14.4 CLI 审批台（qresearch/ui/approvals.py，A1）

```
qresearch approve <project_dir> <plan_id>      # TTY → 计划简报 → y 确认 → approve_plan(actor=HUMAN)
qresearch reject <project_dir> <plan_id>      # 同上（REJECTED + 意见落账）
qresearch conclude <project_dir> <decision_id>  # 需 requires_human；幂等；刷新报告（concluded）
qresearch approvals list <data_root>          # 只读：列出各项目待审批计划
```

_require_tty fail-closed：stdin 非 TTY 一律拒绝（rc=2）——批准这种事只能发生在有人的终端里（本仓库测试 harness 有 pty 属例外，L3 live 走查证）。

审批通道与保真度审计（2026-09-15 增补）：写路径收敛在 ui/approval_ops.py（apply_approve/apply_reject/apply_conclude，幂等；_require_human_channel 结构性拒绝不代表人工在场的通道）。现有两条通道：

通道	在场证明	保真度
TTY（approvals.py）	sys.stdin.isatty() ——agent 子进程结构性拿不到	最高
webui-local（web_approvals.py，qresearch approvals-web）	URL token + session cookie + CSRF，只监听回环	较低：凭据与 agent 同一 OS 用户信任域

每条 approve/reject/conclude 事件把通道写进 detail.channel（core/status.py::ApprovalChannel：tty / webui-local / system-auto / unspecified；历史事件读作"未记录"，不得反推为 tty）。engine.status 的每条决策新增 confirmed_by_human + channel（与 requires_human 区分：前者是"人工真的确认了"）；报告置顶总结把 conclude 通道渲染成独立一行并附保真度说明——"经由较低保真度通道批准"对读报告的人可见，而不是被 actor=HUMAN 抹平。plan_approve（loop）缺省 channel=UNSPECIFIED，auto-approve 落 system-auto。

14.5 M3 运维补齐

- 协作式取消：job_cancel → cancel_requested → experiments_run 的 cancel_check 检查点（准备段：未建账的步骤不再产生记录；执行段：已建账未执行的按取消落账——FAILED，error="CancelledError：人工取消..."，事件标 status=cancelled；已在跑的照常完成——协作式，不杀进程，不谎报）。
- plan_show markdown 卡片：render_plan_markdown(..., edit_hint=False)，含目标量/步骤表/风险/critic 结论与 blocker/diff——给 dsh web 直接渲染。
- adhoc 蒸馏：未验证的 adhoc 记录在蒸馏时生成失败案例层经验（"bash 现算不可引用为证据——要进证据链必须用注册工具跑并 adhoc_verify"）。
- 权限预设（examples/setup_dsh_profile.py）：research-interactive（workspace-write+ask，日常交互） / research-unattended（read-only+ask，无人值守：壳只读，实验仍走引擎子进程不受影响）。注意 patch 的 config 是顶层浅替换——presets 必须带全量 map，改后 dsh --profile research --dump-config 核对（改→验证→记录）。
- 汇报模板：SKILL.md 固定节点汇报模板（轮次/实验/证据/决策建议/下一步），字段全部取自 research_status，禁止 agent 自由发挥。

15. 配置与环境变量

变量/参数	作用	默认
DEEPSEEK_API_KEY	DeepSeek API 凭据（永不打印）	环境提供
QRESEARCH_STATION_TIMEOUT_S	站点轮次看门狗超时	900（15 分钟）
QRESEARCH_DSH_BIN	dsh 可执行文件覆盖	项目 .dsh-runtime/ 内 npm 安装
model（DSHClient）	模型覆盖	deepseek-v4-flash
initialize_timeout_seconds	runtime 启动超时	120
ExperimentManager default_timeout_s	单实验子进程超时	900

16. 数据布局与回放

```
research_data/
├─ dsh_home/                # 全局默认 runtime home（session 持久化）
├─ experiments/<exp_id>/    # 实验产物（inputs.json / result.json / job.sbatch）
├─ demo_phaseN/            # 各阶段离线验收演示产物
├─ demo_phase5_live*/      # P5 live 运行（~25 分钟 3 轮闭环）
└─ demo_phase8_live/       # P8 live 双项目队列（含 proj_a 完整报告与记忆库）
```

回放方法：直接读 events.jsonl（一行一事件），或
EventLog(path).events(project_id=...)；台账用 Storage 列表查询；
报告 report.md 由确定性骨架生成，**发布前需人工复核**（尤其 declare_result
决策无条件人工确认的约定）。

17. 测试与验收矩阵

阶段	机制	离线验收	live 验收
P0 DSH 可行性	SDK + 隔离 runtime	smoke test	smoke test（真实轮次）
P1 台账/恢复	core	resume_demo（roundtrip）	—
P2 规划站点	stations	phase2_demo	真实 LLM 规划全流程（critic 实质物理审查）
P3 实验/工具	experiments+tools	phase3_demo（golden 16/16）	—
P4 三层验证	verification	phase4_demo（注入错误被拦截）	—
P5 研究闭环	research_loop	phase5_demo（3 轮全自动）	PASS ：3 轮 302 事件 terminated，决策链 iterate→replan→terminate
P6 工具构建	tool_builder	phase6_demo	PASS ：tfim_ed 首轮过 golden 8/8 → 晋升

阶段	机制	离线验收	live 验收
P7 研究记忆	memory	phase7_demo（8 项检查）	（prompt 级机制，P5 live 已验闭环）
P8 HPC/ 多项目	hpc+orchestrator	phase8_demo（5 段）	PASS ：双项目队列（1×terminated + 1×needs_human 优雅收尾）

测试：**84 passed**（pytest；分布见 tests/，新增子系统各设独立测试文件）。
已知边界：Slurm 真实提交路径需集群环境验收；quimb 1.15 平台缺陷绕过中。

18. 设计权衡备忘（为什么是这样）

- 1. **六站点而非自由 agent**：自由 agent 的中间态不可回放、不可审计；站点化把 LLM 的贡献压缩到"受 schema 约束的一次提议"，其余全是确定性代码。
- 2. **台账真相在库里**：暂停/恢复/崩溃恢复都从 Storage 重建，而不是内存快照——进程死了不丢状态（P8 live 的 needs_human 项目正是靠它产出部分报告）。
- 3. **验证是出题人驱动的**：golden 数值来自解析/文献/独立实现而非被测实现，"注入 -0.01J 偏移的同名错误实现 → 三层全挂"实测成立。
- 4. **记忆是确定性蒸馏的**：LLM 总结记忆会引入无来源结论；代码蒸馏保证每条教训可回放（代价是表达力有限——换取向实性）。
- 5. **诚实边界优先**：dry-run 不冒充成功、失败实验如实入库、验证 uncertain 不放行、auto-approve 留痕非人工——系统的可信度建立在"不说谎"的机制上而非文档承诺上。